

DATA 201

Data Communication and Ethics

Aye Hninn Khine

Context: The Hidden Layer of Data

Context answers questions like:

- Who collected the data?
- Who is included or excluded?
- Over what time period?
- Under what assumptions?

“The same number can mean different things in different contexts.”

Context Example

Headline:

“Average income increased by 10%”

What might be missing?

- Cost of living
- Income inequality
- Population group
- Time span

Importance of Data Visualization

- **Trend Analysis:** Track changes and patterns over time.
- **Comparisons:** Easily compare categories, groups or metrics.
- **Relationships:** Understand correlations or interactions between variables.
- **Anomaly Detection:** Identify outliers, gaps and unusual behaviors.
- **Decision Support:** Facilitate data-driven decision-making by making insights intuitive.

Bar Charts

Bar charts use rectangular bars to represent numerical values across different categories. The length or height of each bar indicates the magnitude of the data. Bars can be vertical or horizontal.

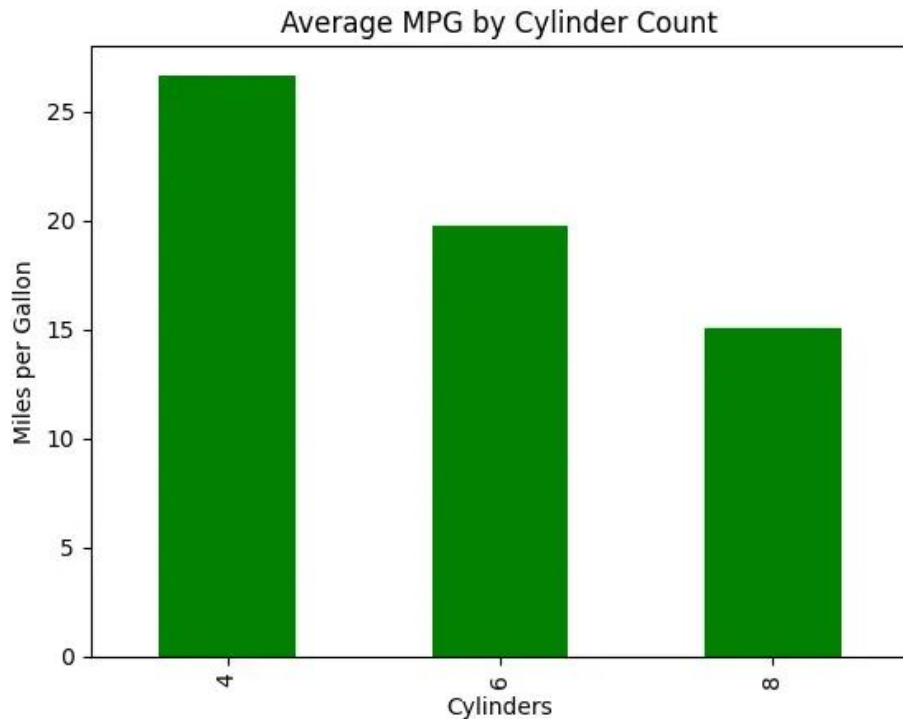
- **Applications:** Comparing average miles per gallon (MPG) across different cylinder counts.
- **Advantages:** Easy to read; effective for comparing multiple categories.
- **Disadvantages:** Can be cluttered with too many categories; misleading if scales differ.

Bar Charts

```
import matplotlib.pyplot as plt
import statsmodels.api as sm

df = sm.datasets.get_rdataset("mtcars").data

df.groupby("cyl")["mpg"].mean().plot(kind="bar",
color="green")
plt.title("Average MPG by Cylinder Count")
plt.xlabel("Cylinders")
plt.ylabel("Miles per Gallon")
plt.show()
```



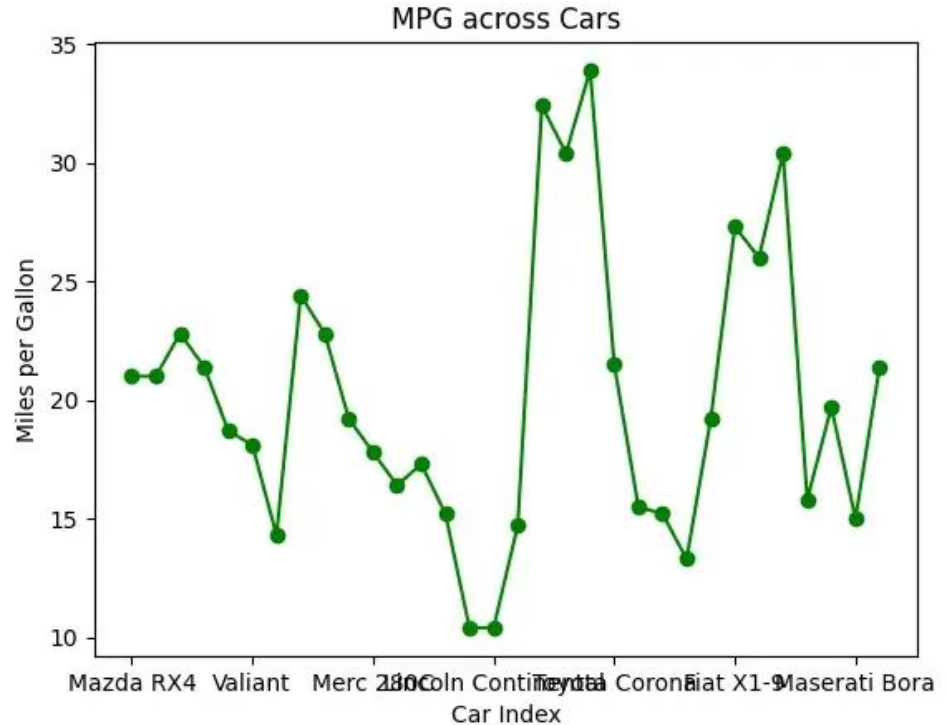
Line Charts

Line charts connect data points with lines across continuous intervals, such as time. They are ideal for displaying trends and patterns over time.

- **Applications:** Showing MPG values across cars (index-wise).
- **Advantages:** Clearly highlights trends over an ordered sequence.
- **Disadvantages:** Not suitable for categorical data; can oversimplify complex patterns.

Line Charts

```
df["mpg"].plot(kind="line", marker="o",  
color="green")  
plt.title("MPG across Cars")  
plt.xlabel("Car Index")  
plt.ylabel("Miles per Gallon")  
plt.show()
```



Pie Charts

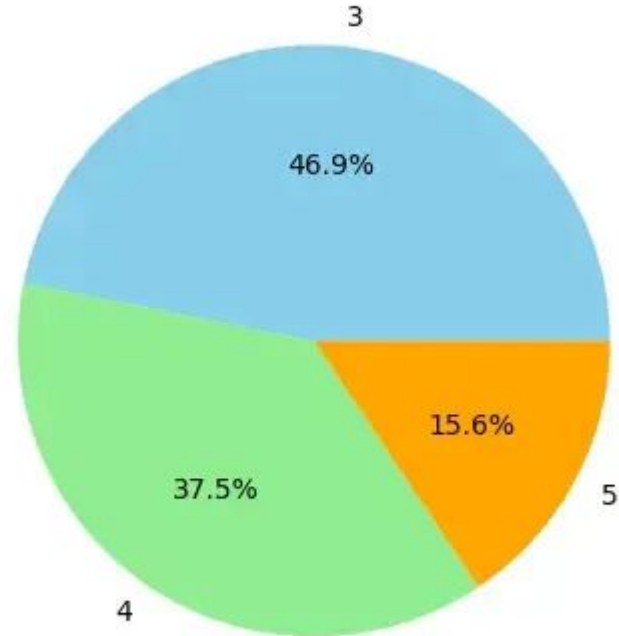
Pie charts display data as proportional slices of a circle, representing each category's contribution to the whole.

- **Applications:** Showing distribution of cars by gear type.
- **Advantages:** Intuitive and visually engaging; shows percentage contribution clearly.
- **Disadvantages:** Difficult to interpret with many categories; slice comparisons may be imprecise.

Pie Charts

```
gear_counts = df["gear"].value_counts()
gear_counts.plot(kind="pie",
autopct="%1.1f%%", colors=[
    "skyblue", "lightgreen", "orange"])
plt.title("Distribution of Cars by Gear Type")
plt.ylabel("")
plt.show()
```

Distribution of Cars by Gear Type



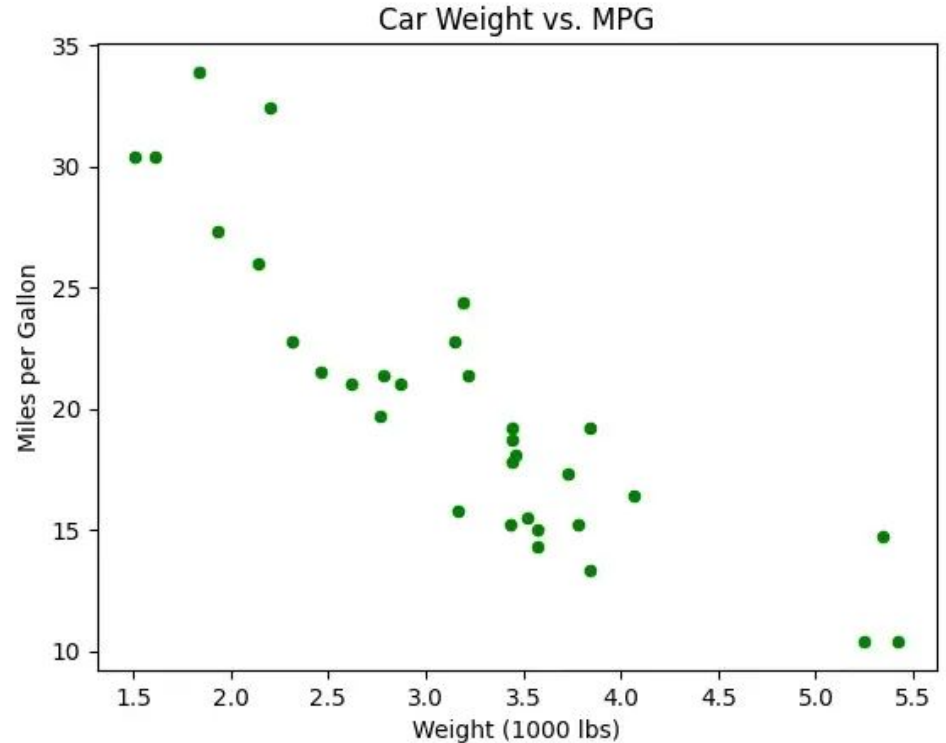
Scatter Plots

Scatter plots represent relationships between two numerical variables using points on a 2D plane. Patterns or correlations can be visually identified.

- **Applications:** Examining relationship between car weight and MPG.
- **Advantages:** Highlights correlations, clusters, and outliers effectively.
- **Disadvantages:** Limited to two variables (unless using size/color to encode a third); not ideal for categorical comparisons.

Scatter Plots

```
df.plot(kind="scatter", x="wt", y="mpg",  
c="green")  
plt.title("Car Weight vs. MPG")  
plt.xlabel("Weight (1000 lbs)")  
plt.ylabel("Miles per Gallon")  
plt.show()
```



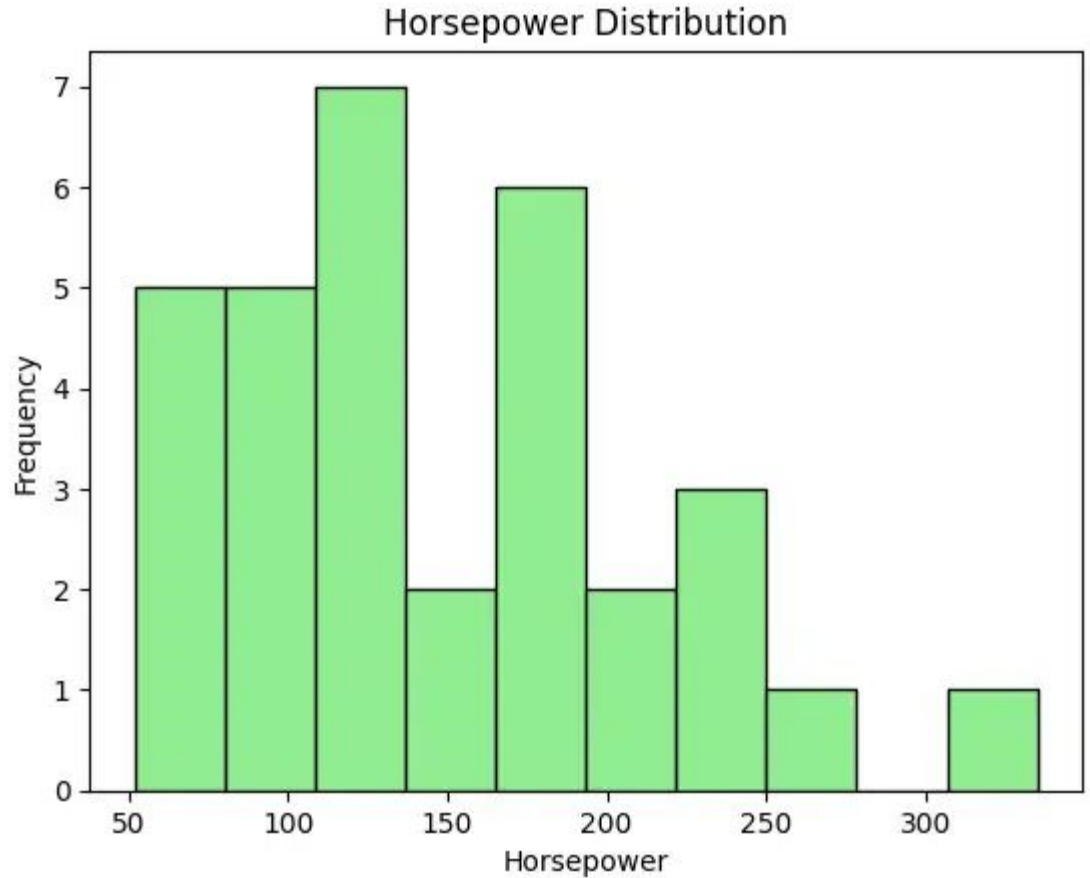
Histograms

Histograms display the distribution of continuous data by grouping values into intervals (bins). They help understand data frequency and spread.

- **Applications:** Understanding horsepower distribution among cars.
- **Advantages:** Shows distribution, peaks, and skewness clearly.
- **Disadvantages:** Less effective for very small datasets; exact values within bins are not visible.

Histograms

```
df["hp"].plot(kind="hist", bins=10,  
color="lightgreen", edgecolor="black")  
plt.title("Horsepower Distribution")  
plt.xlabel("Horsepower")  
plt.show()
```



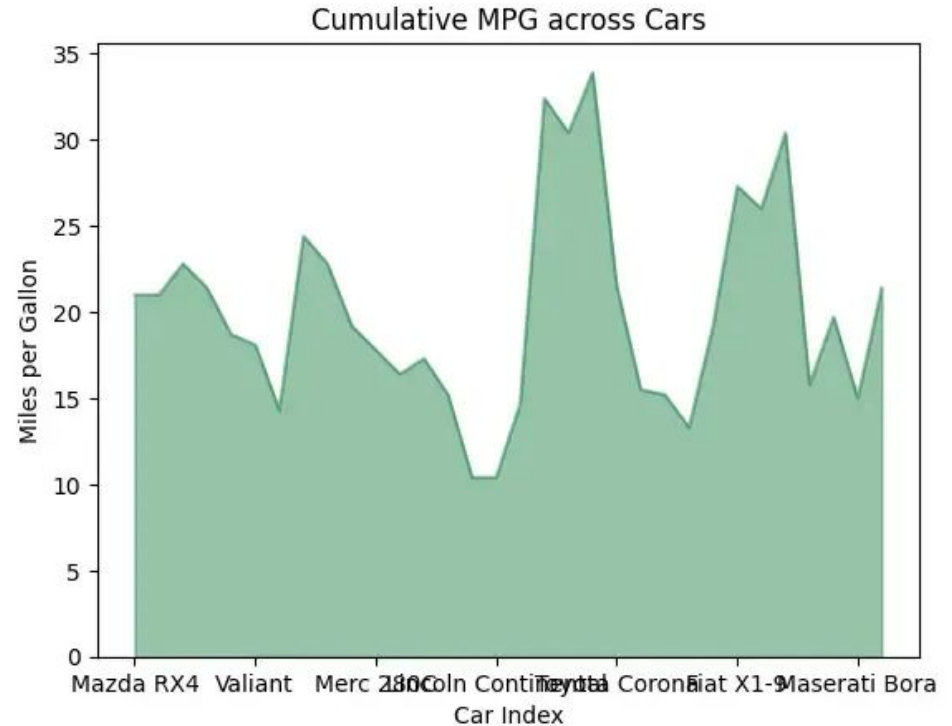
Area Charts

Area charts are line charts with the area beneath filled, emphasizing the magnitude of change over an ordered sequence.

- **Applications:** Showing cumulative MPG across cars.
- **Advantages:** Highlights cumulative trends; visually engaging.
- **Disadvantages:** Can become cluttered with multiple series; less precise for exact values.

Area Charts

```
df["mpg"].plot(kind="area", alpha=0.5,  
color="#2E8B57")  
plt.title("Cumulative MPG across Cars")  
plt.xlabel("Car Index")  
plt.ylabel("Miles per Gallon")  
plt.show()
```



Treemaps

Treemaps display hierarchical data as nested rectangles, with size and color representing values. They are space-efficient and visually powerful for large datasets.

- **Applications:** Visualizing car weights by cylinder counts.
- **Advantages:** Efficient use of space; good for large datasets and proportions.
- **Disadvantages:** Hard to compare exact values visually; layout influences interpretation.

Treemaps

```
import squarify
```

```
sizes = df["wt"]
```

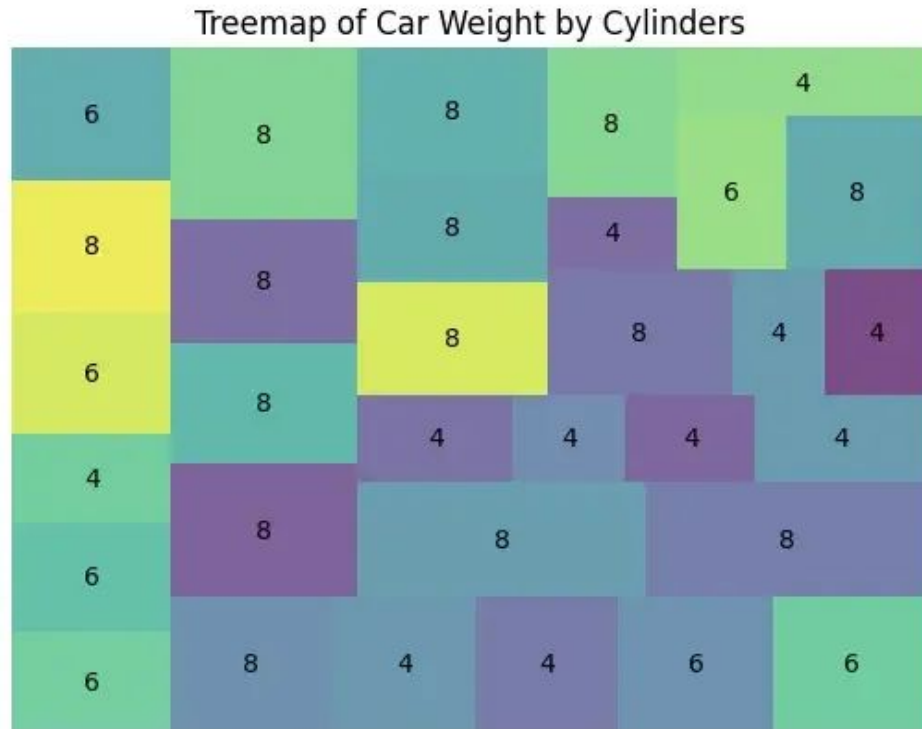
```
labels = df["cyl"].astype(str)
```

```
squarify.plot(sizes=sizes, label=labels,  
alpha=0.7)
```

```
plt.title("Treemap of Car Weight by Cylinders")
```

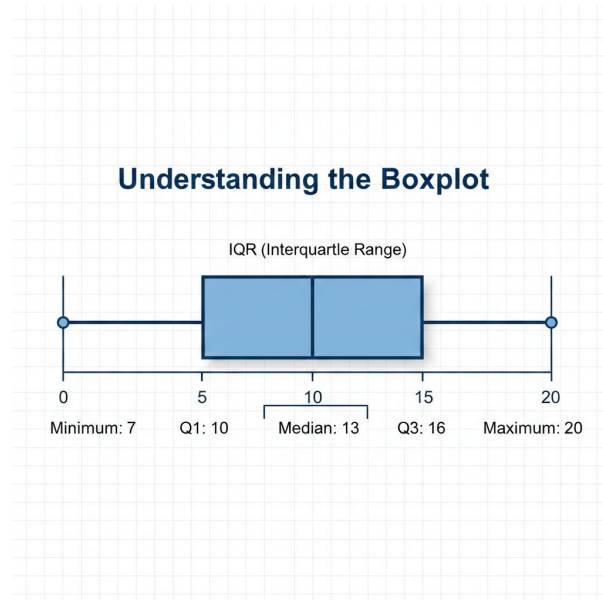
```
plt.axis("off")
```

```
plt.show()
```



Boxplot

A **Box Plot** (or **Whisker plot**) display the summary of a data set, including minimum, first quartile, median, third quartile and maximum. it consists of a box from the first quartile to the third quartile, with a vertical line at the median. the x-axis denotes the data to be plotted while the y-axis shows the frequency distribution.



Boxplot

```
plt.figure()  
plt.boxplot(data, showfliers=True)  
plt.title("Boxplot for Dataset A")  
plt.ylabel("Value")  
plt.show()
```

<https://colab.research.google.com/drive/1XwWBh6FNnUum8oOqnDIspOb6zVR98teP?usp=sharing>

Exploratory Data Analysis in Python

<https://www.geeksforgeeks.org/data-analysis/exploratory-data-analysis-in-python/>

<https://docs.google.com/document/d/1y9DXIncYFYb0AsWvraBiBvq7neoVOi9N5bCbIO5K7vw/edit?usp=sharing>

Thank You

Aye Hninn Khine

ayehninnkhine@parami.edu.mm